

Notion Workspace — build/update specification

Purpose. The Notion MCP connector would not stay connected during the 2026-08-05 session, so this file carries everything Notion needs. Hand it to a Claude session that *does* have Notion access, or work through it by hand. **Do not create new databases.** They already exist — the IDs are below. The job is to *update* them, not duplicate them.

Existing workspace

Workspace **Bonganabob's HQ** · account `hello@bonganabob.com` Hub page: **SokoLink — Mission Control** — `3b3772fd-4e22-8153-8f7d-e305ab5ba18e`

DATABASE	DATA SOURCE ID
Projects (data source named "SOKO LINK")	<code>3b2772fd-4e22-808b-8932-000b69754912</code>
Tasks	<code>3b2772fd-4e22-8066-b77a-000b98667aa5</code>
Docs	<code>3b2772fd-4e22-803c-a78b-000b9b007caf</code>
Decisions	<code>ff8cebf7-0994-4bb5-818a-f4d2ffda97e8</code>
Backlog	<code>6bd6e3e1-7b3b-455c-9e75-ad51619234ca</code>

Tasks schema already has: `Name` (title), `Status` (Not started / Up next / In progress / Done), `Due date`, `Project` (relation), `Day` (number), `Branch` (text), `PR` (url), `Milestone` (checkbox).

What changed and why

The workspace currently describes a **14-day TypeScript build that no longer exists**. On 2026-08-05 the project was re-based:

- **Language changed to Python** (FastAPI/SQLAlchemy/Pydantic). Fredrick is stronger in Python and can debug it unaided.
- **Structure changed to three pillars** — Soko Commerce, Soko Intel, Soko AI.
- **WhatsApp became the interface**, via the in-app browser (not Flows).
- **Project TIKTOK was found** — an existing Python implementation, now a reference.

Step 1 — Rename the `Day` field

Rename Tasks field `Day` → `Milestone No.` (stays a number). The plan is no longer day-based.

Step 2 — Projects database

Update the two existing rows and add one.

ROW	ACTION	NAME	STAGE	TIMELINE
c351e0cc-bd13-4ac5-8fdb-3bd8c40b1f7a	rename	Soko Commerce	In Progress	start 2026-08-06
05a6afb4-5c1b-4935-8b74-2eeade871584	rename	Soko Intel	Planning	—
—	create	Soko AI	Planning	—

Soko Commerce page body:

Selling. Catalog · Orders · Payments · WhatsApp Storefront. A seller's TikTok feed becomes a live catalogue; buyers browse and pay inside WhatsApp. Nothing else matters until a seller can sell.

Soko Intel page body:

Getting seen. Scripts · Captions · Market Insights · Competitor Intelligence. Independent of WhatsApp — can be built in parallel while Meta business verification is pending. The hook corpus is the moat.

Soko AI page body:

The conversation. Customer Support · Sales Agent · Follow-ups · Order Assistance. Sits on top of a working shop. The agent proposes; code disposes.

Step 3 — Tasks database

Delete or archive all 14 existing Day N – **rows**. They describe the retired TypeScript plan.

Create these 10, all Status = `Not started` except P0.

NO.	NAME	PROJECT	BRANCH	MILESTONE
0	P0 — Foundation	Soko Commerce	p0-foundation	☐
1	P1 — Catalog + draft agent	Soko Commerce	p1-catalog	☐
2	P2 — WhatsApp channel	Soko Commerce	p2-wa-channel	☐
3	P3 — Storefront	Soko Commerce	p3-storefront	☐
4	P4 — Orders + Payments	Soko Commerce	p4-payments	☒
5	P5 — Sales agent + support	Soko AI	p5-soko-ai	☐
6	P6 — Follow-ups + order assistance	Soko AI	p6-followups	☐
7	P7 — Competitor intelligence	Soko Intel	p7-intel-insights	☐
8	P8 — Scripts + captions	Soko Intel	p8-intel-generation	☐
9	P9 — Hardening + pilot	Soko Commerce	p9-pilot	☒

Set **P0** to **In progress** .

Task page bodies

Each uses the format: **## Tasks** bullets, **## Done when** , **## Notes** .

P0 — Foundation - FastAPI + SQLAlchemy + Alembic + Pydantic - **Its own Railway Postgres** — the current one holds POC data - Typed config via `pydantic-settings` ; secrets only in `.env` - `/health` with liveness and DB readiness - pytest with transactional fixtures; CI green on every PR - Retire the TypeScript scaffold

Done when: the app boots against its own database and CI is green. *Notes:* One database per application. This rule was learned in TIKTOK 0.2 and nearly re-learned on 2026-08-05 when a shared Railway DB almost got reset.

P1 — Catalog + draft agent - Spike first: re-validate the Apify actor and payload shape - Scraper behind our own adapter; store thumbnails ourselves (TikTok CDN URLs expire) - Seller and Product models with DB-level rails - **The price cascade:** caption hints → cover image → Gemini watches *and listens* - Each video processed **once ever**, keyed by video id, cached

Done when: a real handle yields drafted products with prices read from covers or spoken audio. *Notes:* Captions carry no prices — verified against 24 real captions (0 mention KSh). The price must come from the media. Proven live in TIKTOK at KES 500/550/550/600.

P2 — WhatsApp channel - Cloud API webhook: signature verification, `hub.challenge` , receive - Send text, media, interactive messages, storefront links - **Idempotency at the channel edge** — Meta redelivers - Conversation session state keyed by phone number - **Signed, scoped, short-lived storefront link tokens**

Done when: a real WhatsApp message gets a correct reply, and a replayed webhook changes nothing. *Notes:* Blocked on Meta business verification. Developer account approved.

P3 — Storefront - Server-rendered, mobile-first, minimal client JS - Token-scoped: knows shop and buyer without a login - Catalogue → product detail → selection - Designed empty / error / sold-out states - **Publicly indexable pages** with OG metadata — this is also the SEO surface

Done when: a buyer taps a link in WhatsApp, browses in the in-app browser, and selects a product. *Notes:* Uses the ordinary WhatsApp in-app browser, **not** WhatsApp Flows — no approval dependency, full control of the UI.

P4 — Orders + Payments 🎲 - Order state machine; Kenyan phone normalisation; consent as data - Daraja client: OAuth, STK push, status query - **Idempotent callback** — callback is truth, stock decrements exactly once - Checkout polls order status and survives the STK interruption - Return-to-WhatsApp on completion

Done when: a buyer completes a sandbox M-Pesa payment from the storefront and lands back in the chat confirmed. *Notes:* **Rails before agent.** Built and tested with a plain button before any agent may call it.

P5 — Sales agent + support - Agent grounded **only** in the published catalogue — context injection, not RAG - Sheng-aware; honest about sold-out; never invents a variant - Handoff to the seller when it cannot answer - Tools (`check_stock` , `create_order` , `send_stk_push`) only after P4

Done when: a buyer asks real questions, gets grounded answers, and is guided to a completed purchase.

P6 — Follow-ups + order assistance - Order status on request, answered from real state - Post-purchase follow-up and delivery conversation - Re-engagement / restock: **opt-in only, STOP honoured, paced**; agent drafts, seller approves

Done when: a buyer gets a truthful order answer, and a seller sends an approved restock nudge.

P7 — Competitor intelligence + market insights - **Spike first:** which Apify actor does keyword/niche creator search, its payload, its cost - Keyword → creators in that niche - Benchmarking: followers, views, comments vs the seller's own - Outlier detection - Traction tracking — metrics already in every Apify payload, zero extra cost -

Premium gating enforced in code, per-seat quotas, result caching - **Persist results into the hook corpus from day one**

Done when: a paying seller searches a keyword and sees a ranked benchmarked list — and a repeat search costs nothing. *Notes:* Does **not** depend on Meta. This is the work that keeps moving while verification is pending. Feasibility already confirmed by hand.

P8 — Scripts + captions - **Hook corpus** — hooks from P7's outliers, tagged by niche + language register + performance - Hook and caption generation grounded in the corpus + the seller's real stock - Script generation on the chosen hook - Client reports; weekly digest over WhatsApp - **Feedback loop** — which hooks were filmed, and how they performed

Done when: a seller receives a digest with a hook and script grounded in what works in their niche.

P9 — Hardening + pilot 🎲 - Rate limits, Gemini video cost caps, secrets audit, incident-grade logs - Onboard 2–3 real Kenyan sellers - One real order, end to end

Done when: money moves for real, unattended.

Step 4 — New Decisions rows

Add these to the Decisions database (`Area` , `Status: Active` , `Date: 2026-08-05`).

1. Python, not TypeScript — *Area: Infra / Deploy*

The two-week plan assumed a greenfield TypeScript build. Both assumptions were wrong: a working Python implementation already existed (`Project TIKTOK`), and Fredrick is materially stronger in Python. On a solo build the maintainer's ability to debug unaided outweighs framework elegance. Pydantic replaces Zod and does double duty as the LLM structured-output schema. The TypeScript scaffold built on 2026-08-05 was retired uncommitted.

2. WhatsApp in-app browser, not Flows — *Area: Architecture*

The buyer journey is: WhatsApp chat → browse → **in-app browser opens** → SokoLink storefront → M-Pesa checkout → return to WhatsApp. Chosen over WhatsApp Flows because it removes any dependency on Flows approval, gives full control of the interface, and debugs in an ordinary browser. It also collapses two surfaces into one: the same server-rendered pages serve the in-app storefront *and* public search discoverability. **Three hard parts:** identity across the hop (signed, scoped, short-lived token — never a phone number in a URL); M-Pesa interrupting the browser (checkout polls order status); and the return hop back into the conversation.

3. Grounding, not training — Area: AI / Parsing

Hook generation is grounded on a corpus of real high-performing Kenyan hooks retrieved into the prompt as examples. We are **not** fine-tuning a model. Fine-tuning is expensive, slow, and needs far more data than we will have. Grounding costs a fraction and improves the moment a new hook lands, with no retraining cycle. Revisit only if grounding measurably stops being enough.

4. The hook corpus is the moat — Area: Product

Existing hook tools are not tailored to Kenya — hooks are culture- and language-bound, and English-market "viral formulas" do not transfer. The loop: niche search → outlier posts → extract hooks → corpus → generation grounded in what worked *here*. Every premium search feeds it. A competitor can copy the feature but not the corpus.

5. Niche search is premium-only — Area: Product

It is the only expensive operation a user can trigger repeatedly at will. Three guards, all required: **premium tier** (cost tracks revenue), **per-seat quotas**, **result caching**. This also makes the most expensive feature the one that proves willingness to pay. Commerce acquires sellers free; Intel is the upgrade.

6. Project TIKTOK is a reference, not a foundation — Area: Architecture

Data-Amigo/PROJECT_TIKTOK — 63 commits, 110 passing tests, live on Railway. It proved the hard mechanics: reading a price off a cover, hearing one in Sheng, idempotent Daraja callbacks. Its code is **read and adapted, never copied blind and never reinvented**. Production SokoLink is a different product: WhatsApp is the interface rather than the exit, and Soko Intel does not exist in TIKTOK at all. Modules worth opening first: `services/scraper.py`, `agent/draft.py`, `agent/sales.py`, `services/mpesa.py`, `api/daraja.py`. Its `docs/CONCEPTS.md` and `docs/WORKPLAN.md` are worth reading in full.

Step 5 — Backlog additions

Existing Backlog rows stay. Add:

ITEM	CATEGORY	PRIORITY	SOURCE
Web frontend port to Python templates	Tech debt	Low	2026-08-05
Fine-tuning on Kenyan hooks	Idea	Low	2026-08-05
WhatsApp Flows (native screens)	Idea	Low	2026-08-05

Web frontend port — TIKTOK's Next.js frontend is small (10 routes, 4 components, ~71 kb, three dependencies). Frozen, not ported. Once WhatsApp is the primary surface the web UI's job shrinks. Revisit only if it still earns the work.

Fine-tuning — see the grounding decision. Only if grounding measurably stops being enough.

WhatsApp Flows — the in-app browser makes it unnecessary. Revisit only if Flows would demonstrably beat the webview.

Step 6 — Docs database

Add links to the new documents (all in the repo under `docs/`):

NAME	TYPE
Production Plan (P0–P9)	Strategy
Business Document	Spec
Technical Document — Architecture & Stack	Spec
Ways of Working v2	Guideline

Mark the old **SokoLink — Two-Week Build Plan** as superseded.